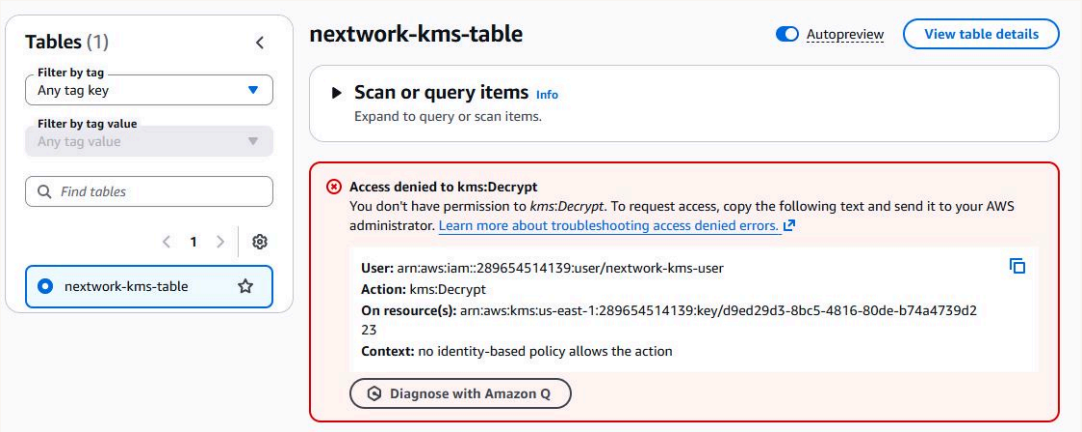
nextwork.org

Encrypt Data with AWS KMS

Mohammed Dawoud Mota



Introducing Today's Project!

In this project I’m here to demonstrate how to securely protect data using AWS KMS and DynamoDB. The goal is to walk through the full process of creating a customer managed KMS key, using that key to encrypt a DynamoDB table, adding data to the table, and then validating that the encryption actually works by testing access with a separate IAM user who doesn’t have permission to decrypt anything. By completing these steps, I’m showing that I understand how AWS handles encryption, key management, and access control, and how these pieces work together to keep data secure.

Tools and concepts

In this project I worked with several AWS services and security concepts that all tie together to protect data in the cloud. The main services I used were AWS KMS for creating and managing my encryption key, DynamoDB as the database I encrypted, and IAM for controlling which users could access or decrypt the data. The key concepts I learned included how encryption protects data at rest, how customer managed keys give me full control over who can use a key, how transparent data encryption allows authorized users to see decrypted data automatically, and how key policies determine whether a user can perform actions like kms:Decrypt. Altogether, the project showed how encryption and access control work together to secure data in a real AWS environment.

Project reflection

It took me a little over an hour to complete the entire project from start to finish. Most of the time went into setting up the KMS key, configuring the DynamoDB table with the right encryption option, and then creating and testing the IAM user to make sure the permissions behaved the way they were supposed to. The actual testing and verification steps were quick once everything was set up correctly.

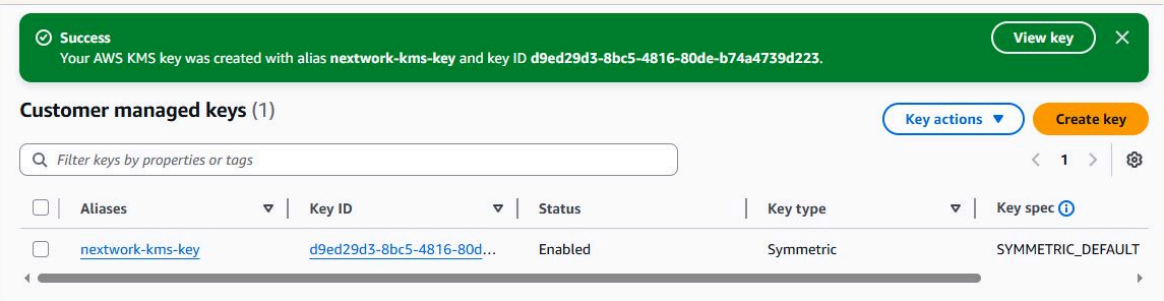
I chose to do this project today because I wanted to learn more about AWS KMS and key management in AWS.

Encryption and KMS

Encryption is the process of taking readable data and transforming it into an unreadable format so that only authorized users can access it. Companies and developers use encryption to protect sensitive information, whether it's customer data, files, or anything stored in a database. It works by using encryption keys that tell the algorithm exactly how to scramble the data into ciphertext. Without the right key, the data just looks like random characters. This is why encryption is such an important part of securing systems and meeting modern security and compliance standards.

AWS KMS is a secure service that acts like a vault for managing encryption keys in the cloud. It lets me create, store, and control the keys that protect data across different AWS services. Instead of handling keys manually or worrying about where they're stored, KMS centralizes everything so I can manage who has access, how the keys are used, and even track every time a key is used through detailed logs. It's designed to make encryption easier and more secure by giving me full control over the keys that protect my data.

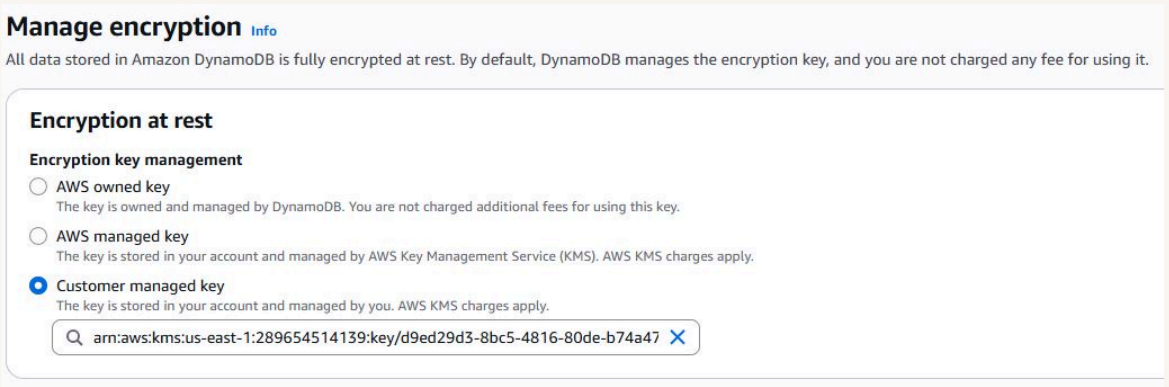
I chose a symmetric key because it uses a single key to both encrypt and decrypt the data, which makes it fast and efficient for securing something like a DynamoDB table. Since this project focuses on encrypting data at rest and keeping the process simple and performant, a symmetric key is the most practical option. Asymmetric keys are better for situations where two different parties need to exchange information securely, but that isn't the case here. For this project, a symmetric key gives me exactly what I need without adding unnecessary complexity.


Success
Your AWS KMS key was created with alias nextwork-kms-key and key ID d9ed29d3-8bc5-4816-80de-b74a4739d223.
View key
Customer managed keys (1)
Key actions
Create key
Filter keys by properties or tags
1
Aliases
Key ID
Status
Key type
Key spec
nextwork-kms-key
d9ed29d3-8bc5-4816-80d...
Enabled
Symmetric
SYMMETRIC_DEFAULT

Encrypting Data

DynamoDB is a fully managed NoSQL database service that lets me store and retrieve data quickly without having to worry about managing servers or infrastructure. It’s built for speed and flexibility, which makes it great for applications that need fast access to large amounts of data. In this project, DynamoDB is the database I’m encrypting with my KMS key, so the key I created will safeguard any data stored in the table.

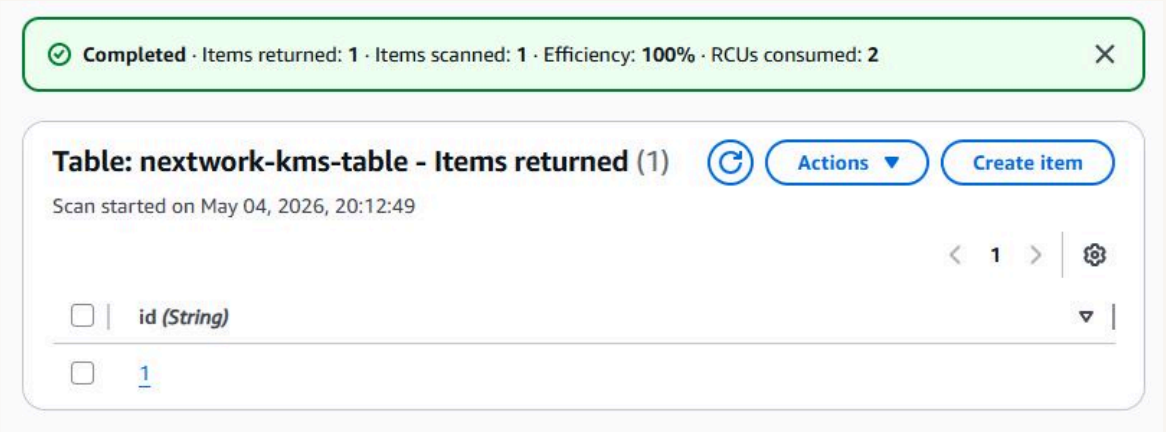
DynamoDB gives me a few different ways to handle encryption, and each option offers a different level of control. The first option is having the encryption key owned by Amazon DynamoDB, where AWS manages everything behind the scenes and I don’t have any visibility into the key itself. The second option is using an AWS managed key, which is still handled by AWS but gives me a bit more visibility into how the key is used. The third option is storing the key in my own account and managing it myself, which is the customer managed key I created in KMS. This last option gives me full control over the key, including who can use it and how it protects the data, which is why it’s the option I used for this project.


Manage encryption Info
All data stored in Amazon DynamoDB is fully encrypted at rest. By default, DynamoDB manages the encryption key, and you are not charged any fee for using it.
Encryption at rest
Encryption key management
AWS owned key
The key is owned and managed by DynamoDB. You are not charged additional fees for using this key.
AWS managed key
The key is stored in your account and managed by AWS Key Management Service (KMS). AWS KMS charges apply.
Customer managed key
The key is stored in your account and managed by you. AWS KMS charges apply.
arn:aws:kms:us-east-1:289654514139:key/d9ed29d3-8bc5-4816-80de-b74a47

Data Visibility

A KMS key manages user permissions through its key policy, which is basically the set of rules that decides who can use the key and what actions they're allowed to perform. When I selected key administrators and key users in the console, AWS automatically translated those choices into policy statements behind the scenes. The key policy is what ultimately controls access, so even if someone has permissions in IAM, they still can't use the key unless the key policy allows it. This is why my test user, who had full DynamoDB access, still couldn't decrypt anything. The KMS key simply didn't grant them permission to perform actions like kms:Decrypt, and without that, the encrypted data stays protected.

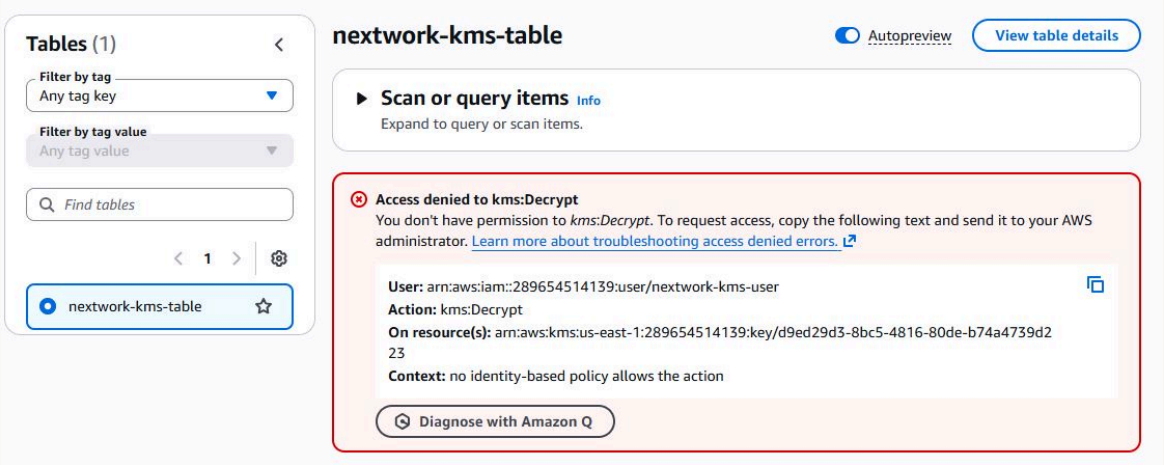
The table's items are visible to me because I have the correct permissions to use the KMS key that encrypts the data. Even though the data is stored in an encrypted format, DynamoDB automatically decrypts it on my behalf since I'm an authorized user. This is called transparent data encryption, and it allows the system to keep the data secure at rest while still letting someone with the right access view it in a readable form.


Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2
Table: nextwork-kms-table - Items returned (1)
Actions
Create item
Scan started on May 04, 2026, 20:12:49
1
id (String)
1

Denying Access

I set up my IAM user by creating a new user specifically for testing access to the encrypted DynamoDB table. I gave this user permission to sign in to the AWS Management Console and then attached the AmazonDynamoDBFullAccess policy so they could fully interact with DynamoDB. At the same time, I made sure not to give them any permissions related to the KMS key I created earlier. This setup allowed me to test what happens when a user has full DynamoDB access but no ability to decrypt data protected by my customer managed KMS key.

When I checked the DynamoDB table as the test user, I wasn't able to see any of the items at all. Instead of the data, I got an access denied message explaining that I didn't have permission to use the KMS key to decrypt the table. Even though the user had full DynamoDB access, the encryption blocked them because they weren't authorized to perform kms:Decrypt. That confirmed that the KMS key was working exactly the way it should by protecting the data from anyone who doesn't have the right permissions.

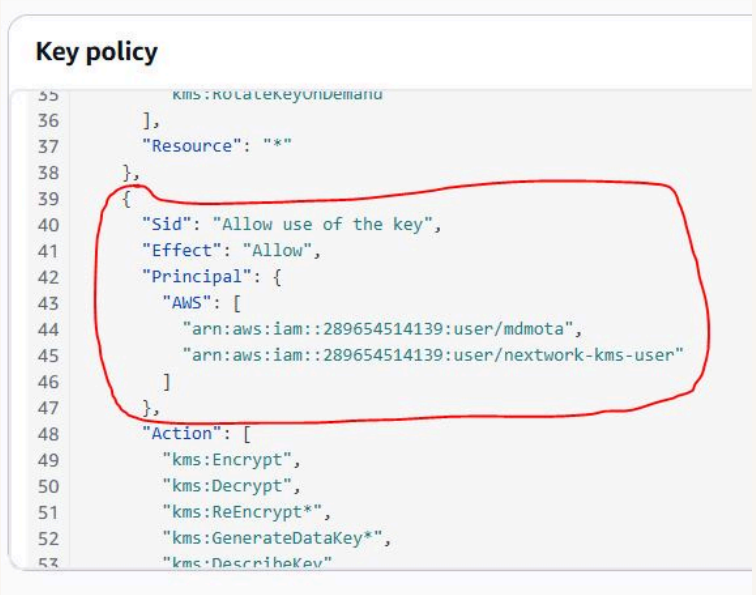

Tables (1)
Filter by tag
Any tag key
Filter by tag value
Any tag value
Find tables
1
nextwork-kms-table
nextwork-kms-table
Autopreview
View table details
Scan or query items Info
Expand to query or scan items.
Access denied to kms:Decrypt
You don't have permission to kms:Decrypt. To request access, copy the following text and send it to your AWS administrator. Learn more about troubleshooting access denied errors.
User: arn:aws:iam::289654514139:user/nextwork-kms-user
Action: kms:Decrypt
On resource(s): arn:aws:kms:us-east-1:289654514139:key/d9ed29d3-8bc5-4816-80de-b74a4739d223
Context: no identity-based policy allows the action
Diagnose with Amazon Q

EXTRA: Granting Access

I granted my user permission to use the key by updating the key policy on my customer managed KMS key. As the administrator, I went back to the key in the KMS console and modified its policy to include the test user, giving them the ability to perform actions like kms:Decrypt. Once the policy was updated, the user was finally able to access and decrypt the data in the DynamoDB table. This confirmed that the key policy is the deciding factor for who can actually use the key, even if someone already has full DynamoDB permissions.

I verified my test user's access by logging into the AWS console using the sign-in URL and credentials I created for that user. Once I was in their account, I went to the DynamoDB console, opened the Explore items section, and selected the encrypted table. After updating the key policy earlier, the user was finally able to view the table's items without getting the access denied message. Seeing the decrypted data appear confirmed that the key policy change worked and that the user now had permission to use the KMS key to decrypt the table's contents.

You would use encryption when you need to protect the actual data itself, even if someone manages to access the system where it's stored. Encryption makes the data unreadable without the right key, so it adds a deeper layer of security beyond just permissions. Access control tools like IAM decide who is allowed to interact with a resource, but they don't protect the data if someone bypasses those controls or gains access in another way. Encryption is what keeps the data safe at rest, while access control manages who can request or use it. In a secure environment, both work together, but encryption is what ensures the data stays protected no matter what.


Key policy
35 kms:RotateKeyOnDemand
36],
37 "Resource": "*"
38 },
39 {
40 "Sid": "Allow use of the key",
41 "Effect": "Allow",
42 "Principal": {
43 "AWS": [
44 "arn:aws:iam::289654514139:user/mdmota",
45 "arn:aws:iam::289654514139:user/nextwork-kms-user"
46]
47 },
48 "Action": [
49 "kms:Encrypt",
50 "kms:Decrypt",
51 "kms:ReEncrypt*",
52 "kms:GenerateDataKey*",
53 "kms:DescribeKey"

The place to learn & showcase your skills

Check out nextwork.org for more projects